

项目概述

本项目基于 DVWA (Damn Vulnerable Web Application) 靶场环境，在 Kali Linux 平台上完成 SQL 注入漏洞的渗透测试实战。项目覆盖 DVWA 安全级别中的 Low 和 Medium 两个级别，旨在完整复现 SQL 注入漏洞的发现、验证、利用及信息枚举全过程，并输出详细的技术报告。

测试环境

操作系统：Kali Linux

Web 服务器：Apache 2

数据库：MariaDB

靶场系统：DVWA

测试工具：Firefox 浏览器、Burp Suite 社区版

测试时间：2026 年 7 月

报告结构说明

本报告共包含四个部分。第一部分为 Low 级别 SQL 注入复现，包含从正常查询基线建立到通过 information_schema 枚举数据库结构的完整流程。第二部分为 Medium 级别 SQL 注入复现，重点展示如何通过 Burp Suite 抓包改

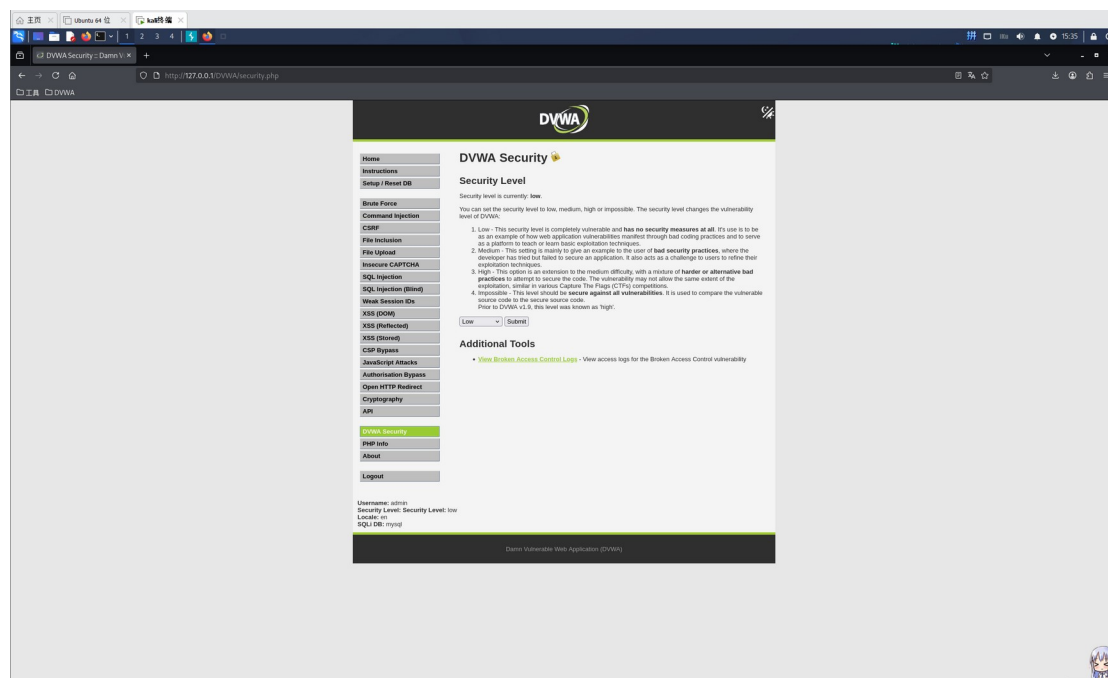
包的方式绕过前端下拉菜单限制，实现与 Low 级别相同的注入效果。第三部分为 Low 与 Medium 级别的差异对比分析。第四部分为本次实战的技术总结与防御建议。

第一部分为 Low 级别 SQL 注入复现

第一步：确认安全级别为 Low

图 1.安全级别设计为 Low

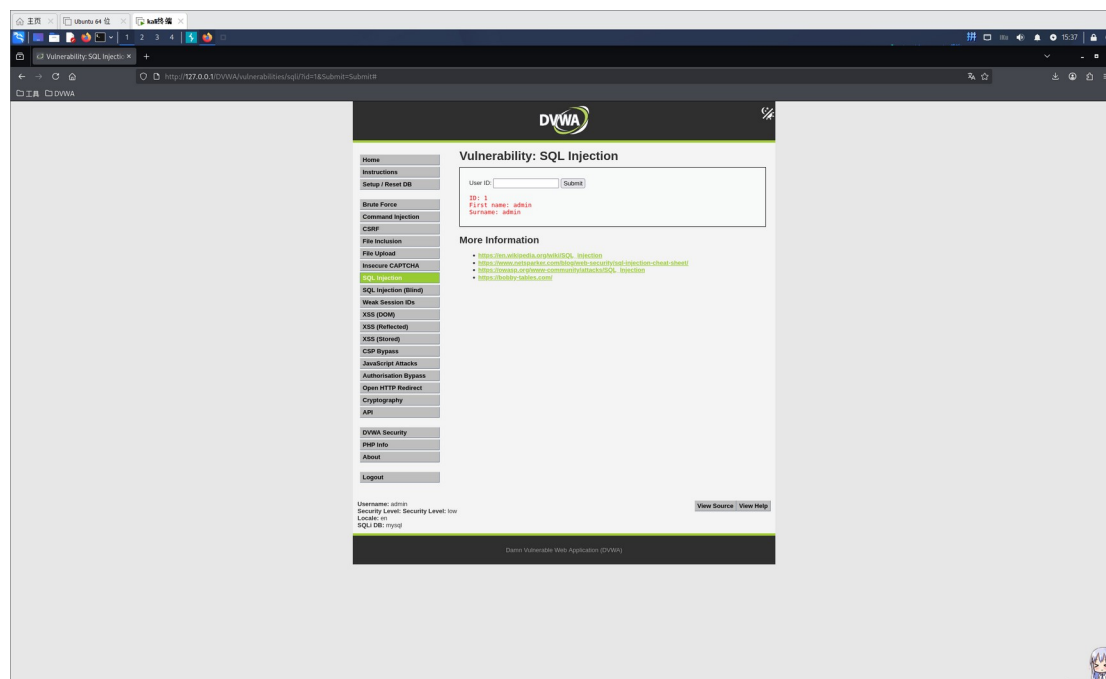
原理分析：DVWA 的安全级别分为 Low、Medium、High 和 Impossible。Low 级别不对用户输入做任何过滤或转义，直接将输入拼接到 SQL 语句中执行。这一步的目的有两个：一是确认测试环境符合预期，二是建立“存在漏洞”的基线前提。如果安全级别是 Medium 或 High，同样的 Payload 可能无法生效，因此必须在开始测试前确认



第二步：正常查询（建立基线）

图 2：正常输入 1，返回 admin 用户信息

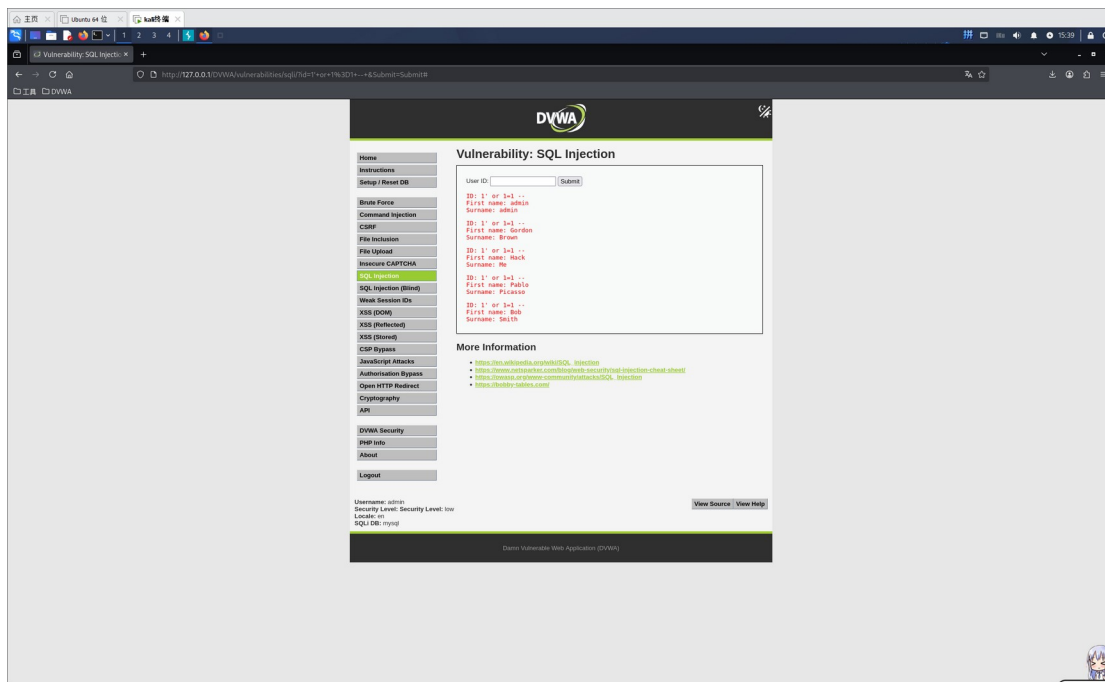
原理分析：输入正常值 1，观察页面返回的内容。这一步的目的是建立“正常行为”的对照基线。当页面返回 First name: admin, Surname: admin 时，说明这个输入对应的是 users 表中 user_id=1 的记录。后续所有注入操作的效果，都需要和这个“正常状态”做对比——如果注入后返回的内容变多了、变少了、或出现了预期外的数据，就说明注入成功。没有基线，就无法判断注入是否生效



第三步：验证注入存在（证明漏洞）

图 3：输入 1' or 1=1 --，返回所有用户，证明注入存在

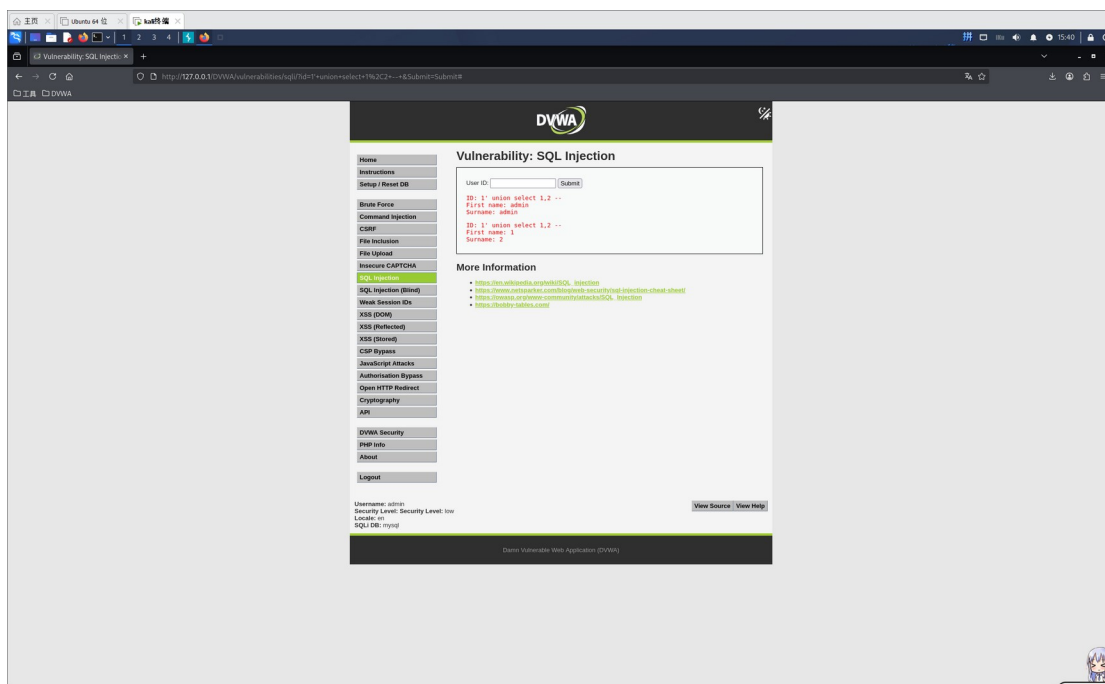
原理分析：输入 1' or 1=1 -- 时，后台 SQL 语句变为：SELECT first_name, last_name FROM users WHERE user_id = '1' and 1=1 --。1=1 是恒真条件，使得整个 WHERE 子句永远为真，因此数据库返回了 users 表中的所有记录。这一步的核心价值在于：它证明了 user_id 参数存在 SQL 注入漏洞。攻击者可以通过构造特定的输入，改变 SQL 语句的原本逻辑。同时，-- 是 MySQL 的注释符，它将原本语句末尾的单引号注释掉，避免了语法错误，这是 SQL 注入中闭合与注释的典型技巧



第四步：判断列数（为 UNION 做准备）

图 4：判断列数为 2，UNION 查询可用【1' union select 1,2 --】

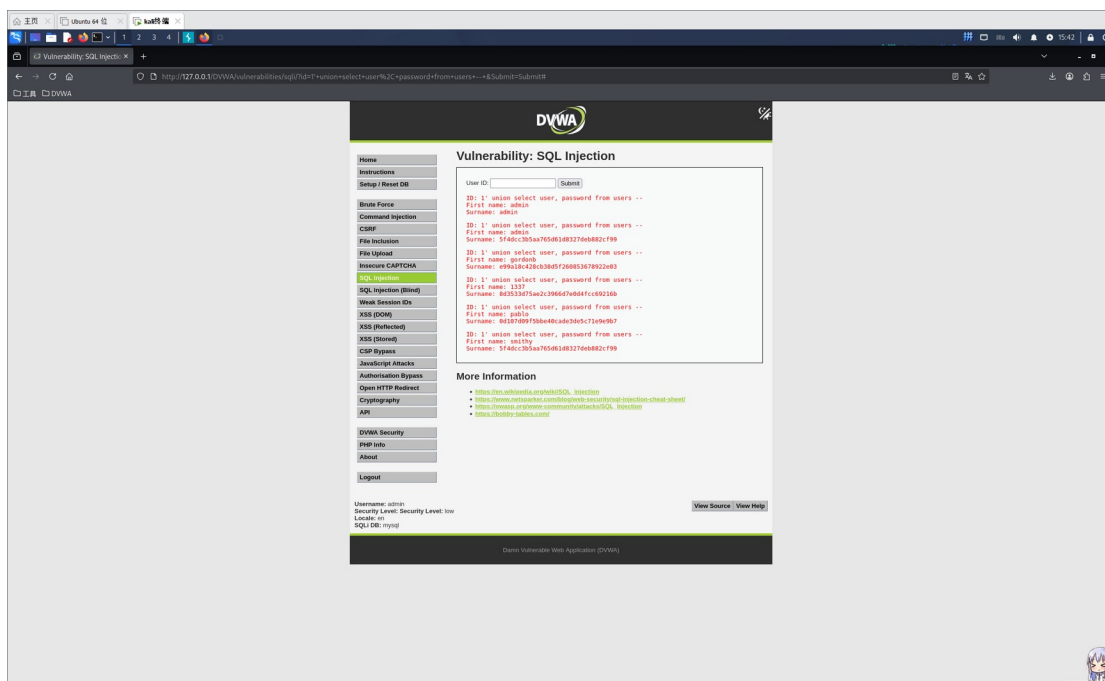
原理分析：输入 1' union select 1,2 -- 时，UNION 关键字用于合并两条 SELECT 语句的查询结果。但 UNION 有一个严格的要求：前后两条 SELECT 语句必须返回相同数量的列。页面返回了数字 1 和 2，说明原始查询返回了 2 列。这一步是 UNION 注入的前提——不知道列数，就无法构造正确的 UNION 查询。这好比要拼接两块积木，必须先知道每一块有几个接口



第五步：拖出所有用户名和密码哈希（核心成果）

图 5：通过 UNION 注入拖出 users 表所有用户名和密码哈希【1' union select user, password from users --】

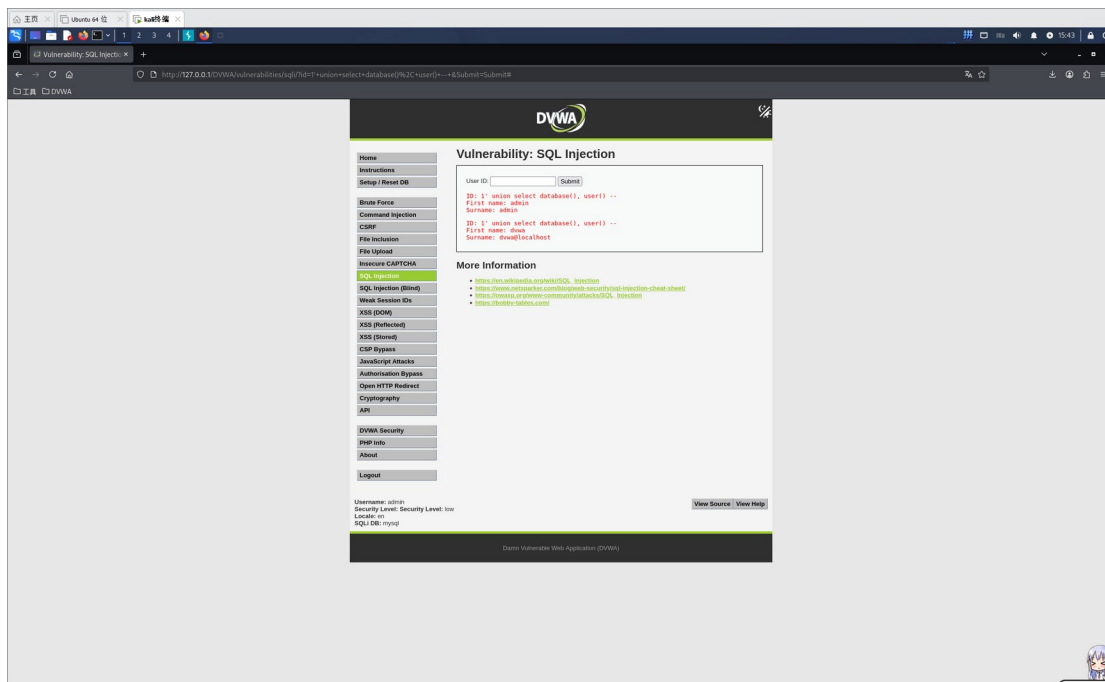
原理分析：输入 1' union select user, password from users -- 时，UNION 右侧是攻击者构造的查询：从 users 表中选取 user 和 password 列。由于原始查询返回 2 列，攻击者的查询也返回 2 列，满足 UNION 的合并条件，因此数据库将两张表的结果合并后一起返回。页面中显示的 password 列是 MD5 哈希值，而非明文密码——这是数据库安全存储密码的常见做法。但 MD5 是单向哈希，不等于不可逆，实际渗透中常用彩虹表或暴力破解手段还原明文。这一步是整个 SQL 注入流程中的核心成果：攻击者通过一次注入，获取了数据库中所有用户的凭证信息



第六步：查询当前数据库名

图 6：探测当前数据库名和连接用户【1' union select database(), user() --】

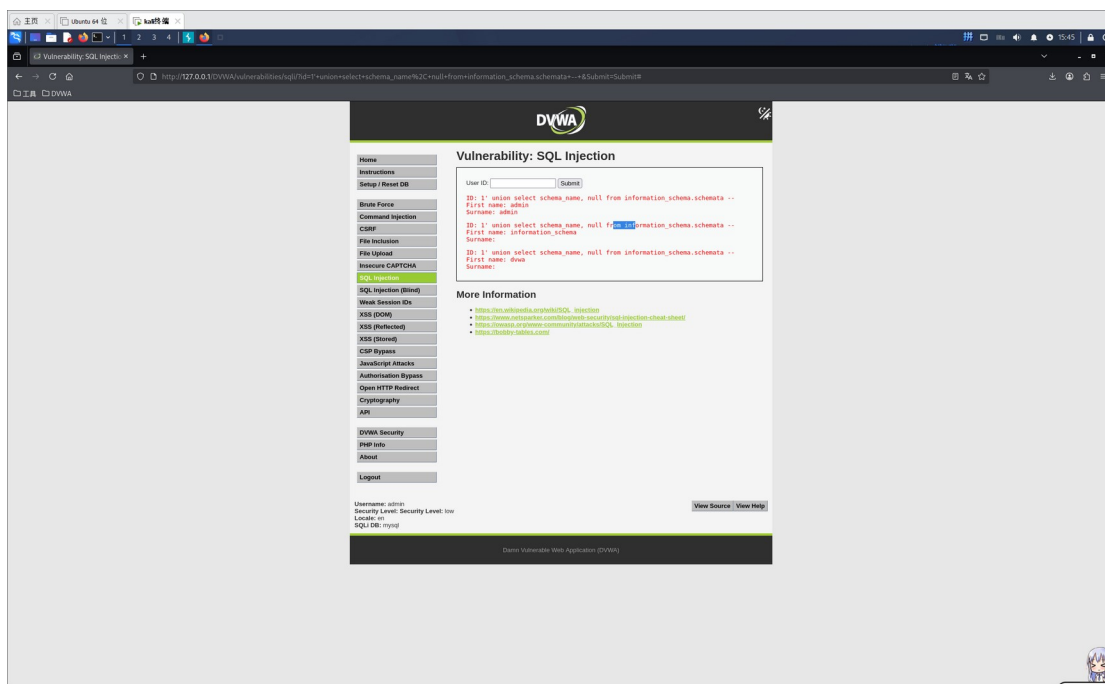
原理分析：输入 1' union select database(), user() -- 时，database() 和 user() 是 MySQL 的内置函数，分别返回当前数据库名和当前连接的用户名。这一步的目的是获取环境信息。在实际渗透中，攻击者往往需要先确认当前所在数据库的名称和连接账户的权限级别，这决定了后续能执行哪些操作。例如，如果当前用户是 root，攻击者可能进一步尝试提权或写 Webshell。信息探测是渗透测试中“侦察”阶段的关键环节



第七步：查询所有数据库名

图 7：通过 information_schema 枚举所有数据库【1' union select schema_name, null from information_schema.schemata --】

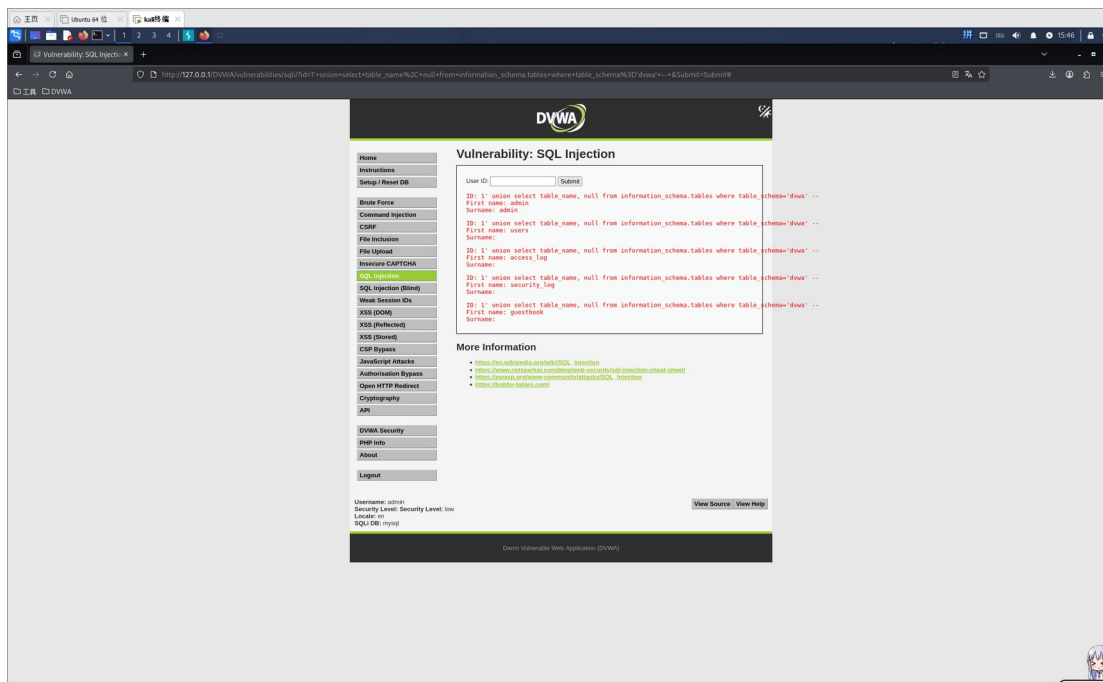
原理分析：输入 `1' union select schema_name, null from information_schema.schemata --` 时，目标表是 `information_schema.schemata`。`information_schema` 是 MySQL 自带的一个系统数据库，存储了所有数据库的元信息，包括数据库名、表名、列名等。`schemata` 表记录了所有数据库的名称。这一步的本质是：通过注入点查询系统表，获取整个数据库服务器的数据库列表。实际渗透中，攻击者通过这一步可以发现高价值目标，如用户库、订单库、配置库等，为后续深入渗透提供方向。



第八步：查询当前数据库所有表名

图 8：枚举 dvwa 数据库中的所有表【1' union select table_name, null from information_schema.tables where table_schema='dvwa' --】

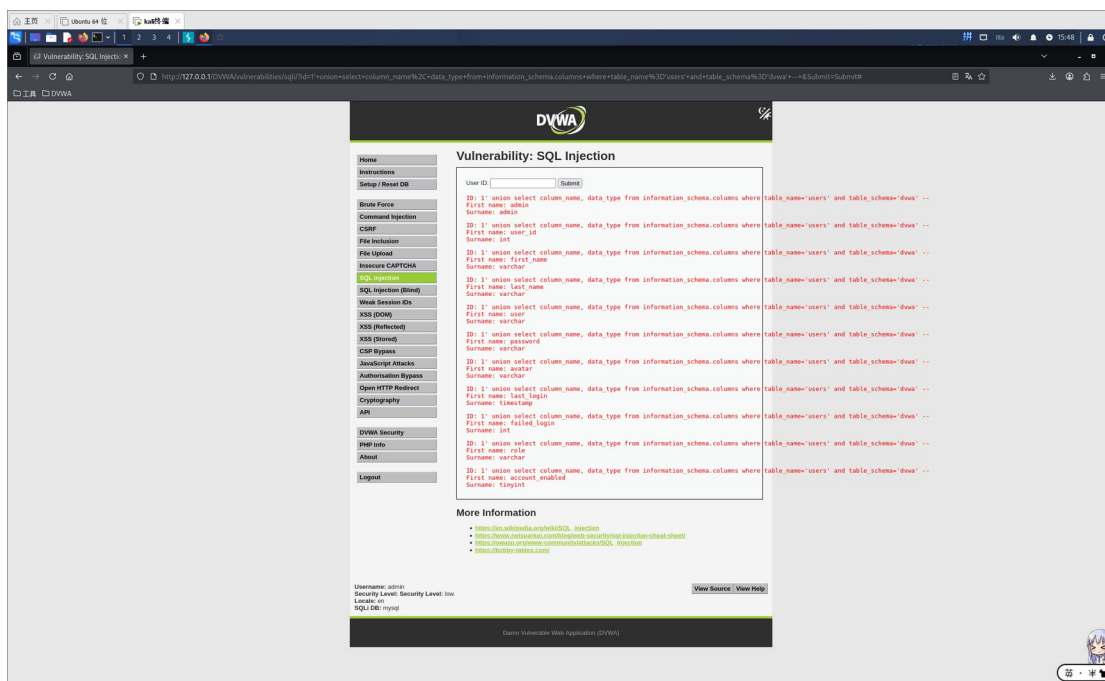
原理分析：输入 1' union select table_name, null from information_schema.tables where table_schema='dvwa' -- 时，目标表是 information_schema.tables，它记录了所有数据库中所有表的信息。where table_schema='dvwa' 将结果限制在当前数据库 dvwa 内。返回结果包括 users、guestbook、access_log、security_log 等表名。这一步的价值在于：攻击者通过枚举表名，可以判断数据库中存储了哪些类型的数据。比如看到 users 表，就知道里面可能存有用户凭证；看到 orders 表，就知道可能存有订单或支付信息



第九步：查询 users 表的列结构

图 9：查看 users 表的结构，确认 user 和 password 列存在【1' union select column_name, data_type from information_schema.columns where table_name='users' and table_schema='dvwa' --】

原理分析：输入 1' union select column_name, data_type from information_schema.columns where table_name='users' and table_schema='dvwa' -- 时，目标表是 information_schema.columns，它记录了所有数据库中所有表的列信息。where table_name='users' and table_schema='dvwa' 精确锁定 dvwa 库中的 users 表。返回结果包括 user_id、first_name、last_name、user、password 等列名及其数据类型。这一步是信息收集中最关键的一环：通过获取目标表的列结构，攻击者可以精确构造后续的 SELECT 查询，提取所需的数据。例如知道 user 和 password 列存在后，才能写出第五步中的 select user, password from users

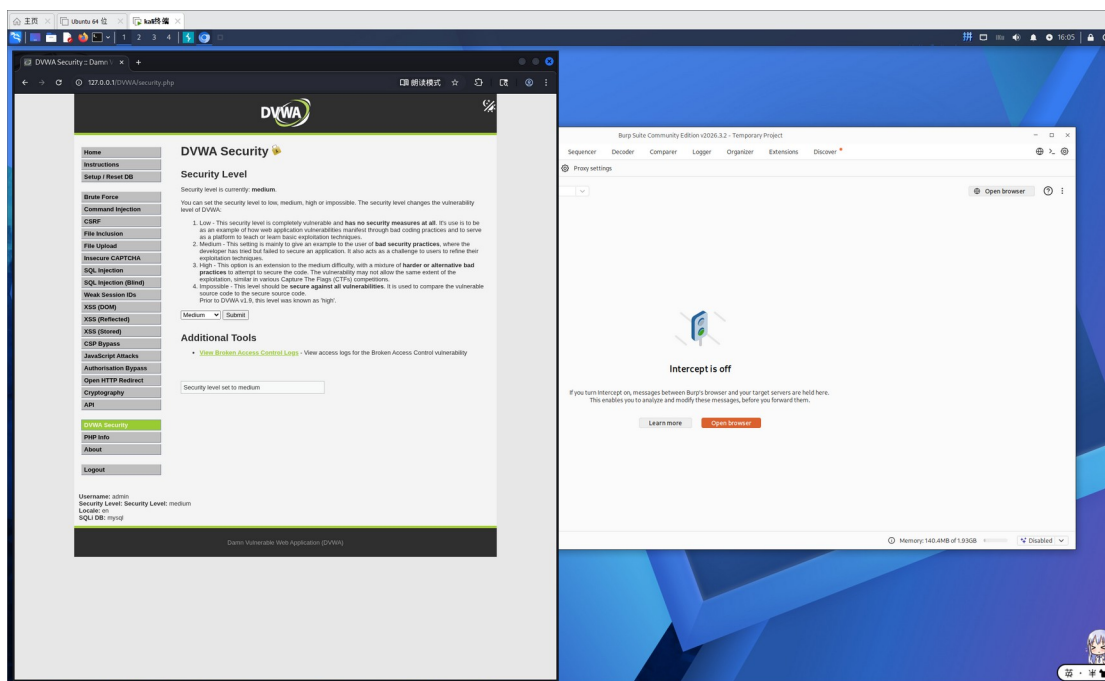


第二部分为 Medium 级别 SQL 注入复现，重点展示如何通过 Burp Suite 抓包改包的方式绕过前端下拉菜单限制，实现与 Low 级别相同的注入效果

第 1 步：切换安全级别为 Medium

图 1：安全级别设为 Medium

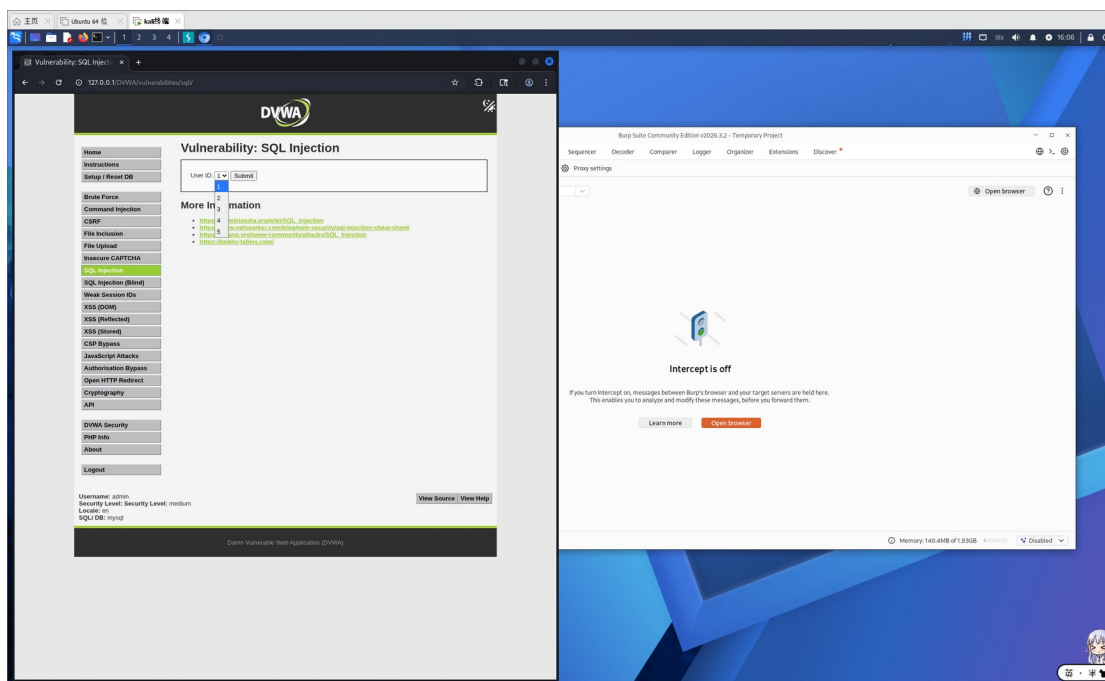
原理分析：Medium 级别引入了前端限制，将输入框替换为下拉菜单，试图通过限制用户输入来防止注入。但前端验证仅作用于浏览器端，无法防御攻击者通过代理工具篡改请求。这一步的目的是确认测试环境切换到 Medium 级别，为后续注入做准备



第2步：进入 SQL Injection 模块

图 2：Medium 级别将输入框替换为下拉菜单，前端限制用户输入

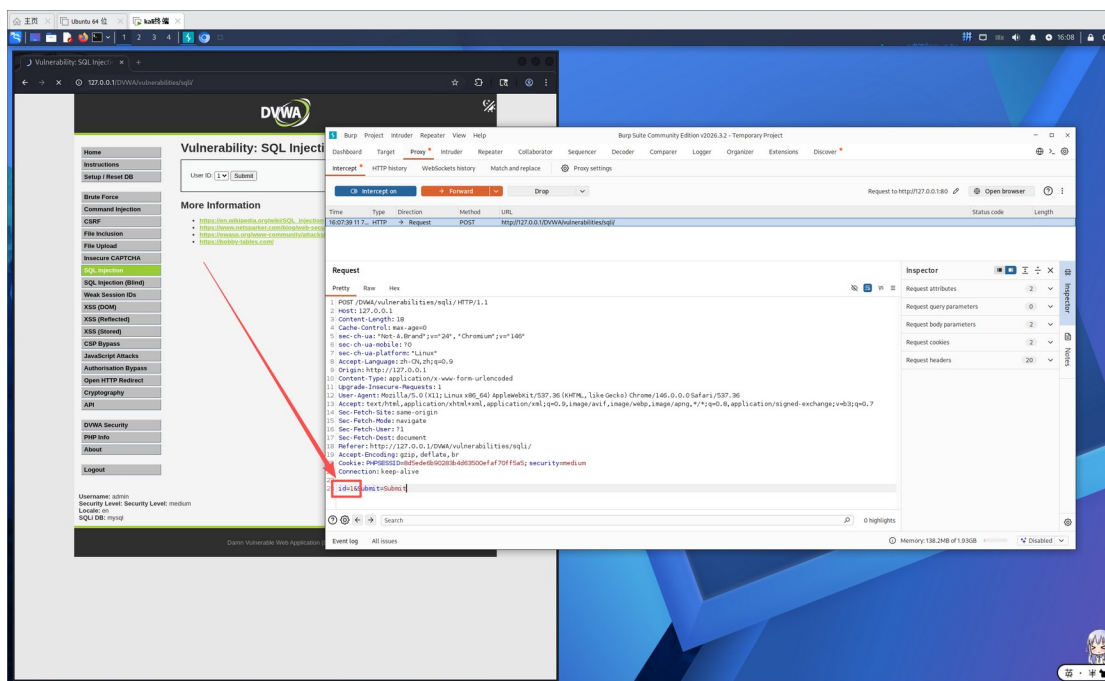
原理分析：进入 SQL Injection 模块后，页面显示下拉菜单，选项为 1 到 5，用户无法直接输入任意值。这种设计基于一个错误的安全假设：即用户只能通过浏览器界面与服务器交互。实际上，攻击者可以使用 Burp Suite 等中间人代理工具，在请求到达服务器之前修改其内容



第 3 步：Burp 抓取下拉菜单的请求

图 3：Burp 拦截到下拉菜单提交的原始请求，id=1

原理分析：Burp Suite 作为中间人代理，拦截了浏览器发送的 HTTP 请求。通过分析请求报文，可以发现 id 参数位于 URL 的 GET 请求中，值为 1。这一步的关键在于：无论前端如何限制用户输入，最终传递给后端的都是 HTTP 请求。攻击者可以通过篡改请求中的参数值，达到与自由输入相同的效果



第 4 步：修改请求参数，注入验证

原理分析：在 Burp 中将 id 参数从 1 修改为 1 or 1=1 并放行请求。由于 Medium 级别的 SQL 语句为 `SELECT first_name, last_name FROM users WHERE user_id = '$id'`，且 id 被单引号包裹，但数字型注入不需要闭合单引号。因此 1 or 1=1 被拼接后变成 `WHERE user_id = '1' or 1=1`，由于 1=1 恒为真，数据库返回了所有用户记录。这一步验证了 Medium 级别仍然存在 SQL 注入漏洞。尽管存在 `mysql_real_escape_string` 转义函数，但该函数只转义单引号，对于不带引号的数字型 Payload 完全无效

图 4A：Burp 中将 id 参数修改为注入 1 and 1=1

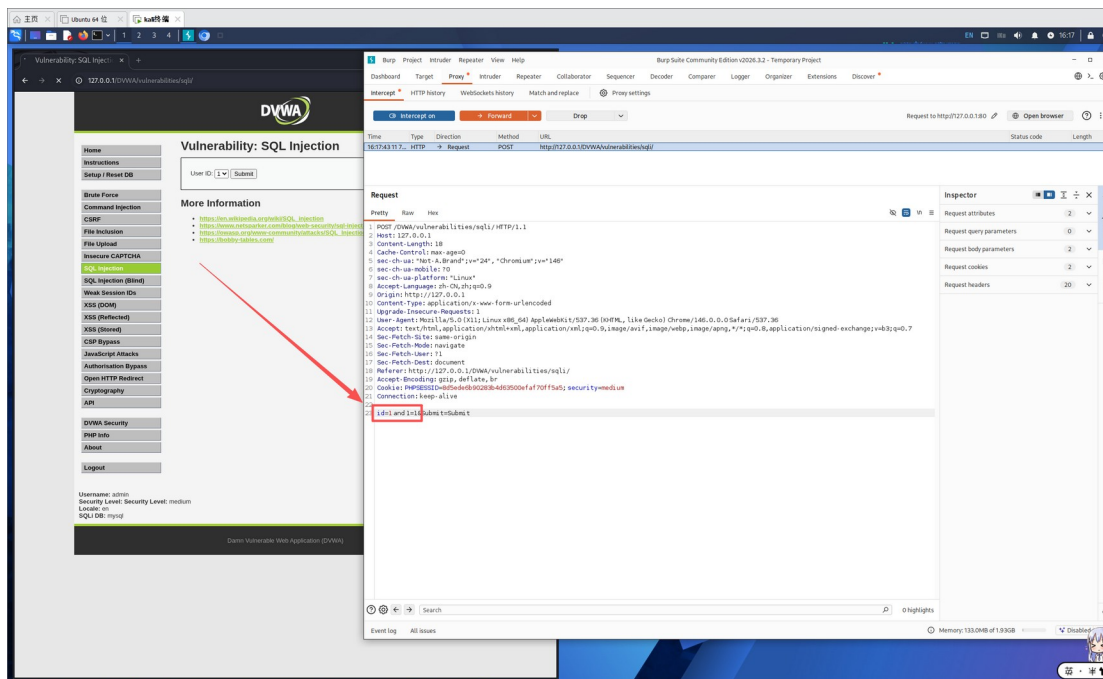
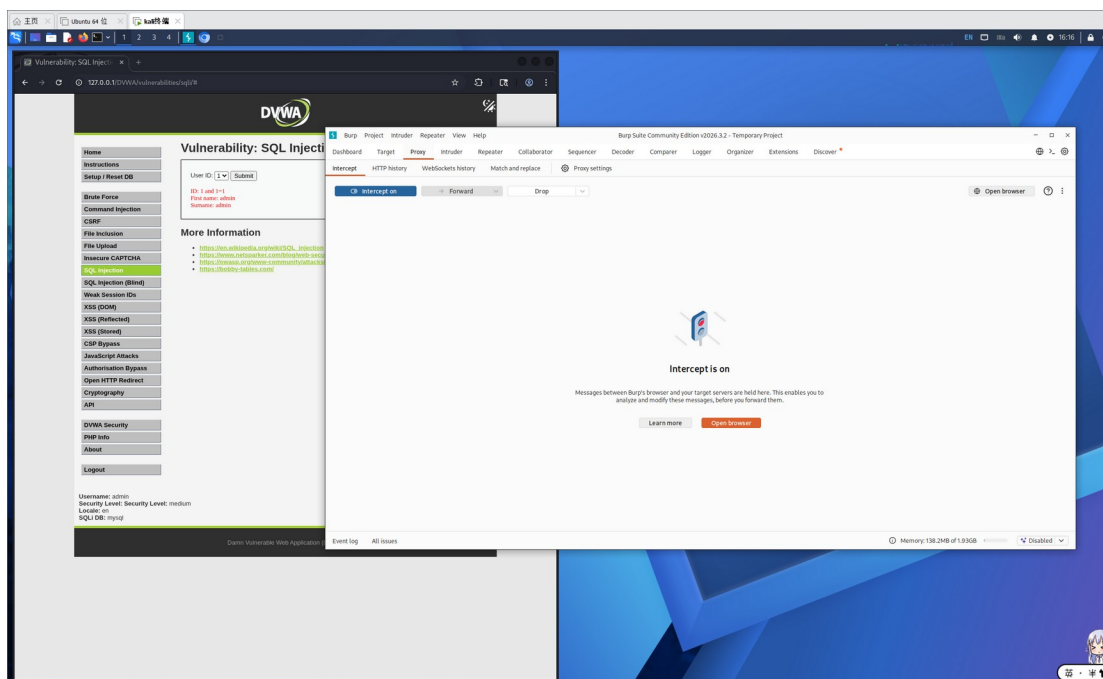


图 4B：返回所有用户，证明 Medium 级别仍然存在 SQL 注入漏洞



第 5 步：UNION 注入拖数据

原理分析：在 Burp 中将 id 参数修改为 1 union select user, password from users。由于前端下拉菜单限制被 Burp 绕过，该请求成功到达服务器。数据库执行了攻击者构造的 UNION 查询，将

users 表中的所有用户名和密码哈希与原始查询结果合并并返回。由于 Low 级别已确认该表为 2 列结构，且 Medium 级别查询的是同一张表，列数相同，因此无需重复判断列数，直接使用 UNION select user,password from users

图 5A : Burp 中将 id 参数修改为 UNION 拖数据 Payload

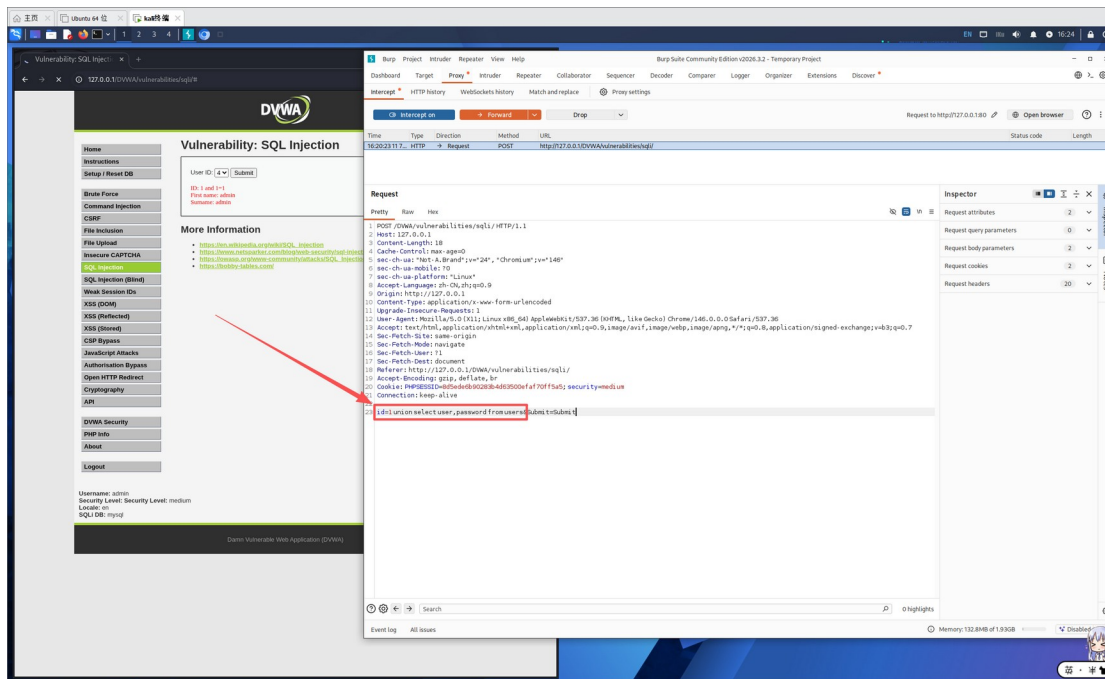
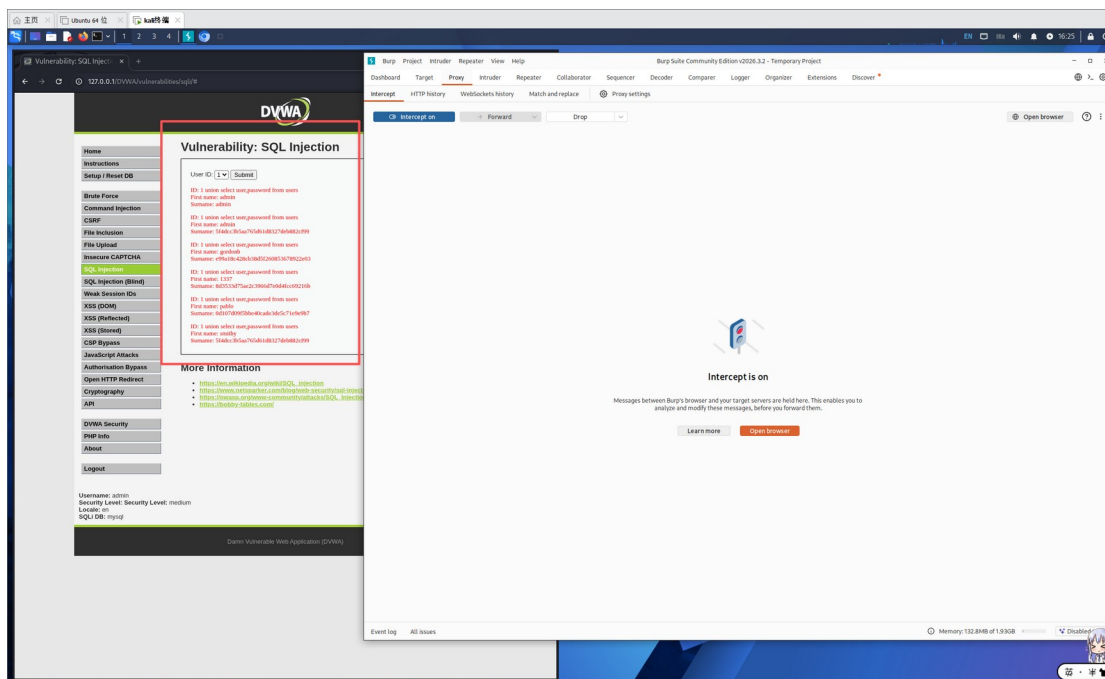


图 5B : 成功拖出 users 表所有用户名和密码哈希



第六步：查询当前数据库名和连接用户

原理分析：在 Burp 中将 id 参数修改为 1 union select database(), user()。通过 MySQL 内置函数 database() 和 user() 获取当前数据库名和连接用户名。这一步在渗透测试中属于信息收集阶段，攻击者需要确认当前数据库环境，以决定后续攻击路径。例如，如果当前用户权限较高，可能进一步尝试提权或访问其他数据库

图 6A：Burp 中修改请求为查询当前数据库名和连接用户

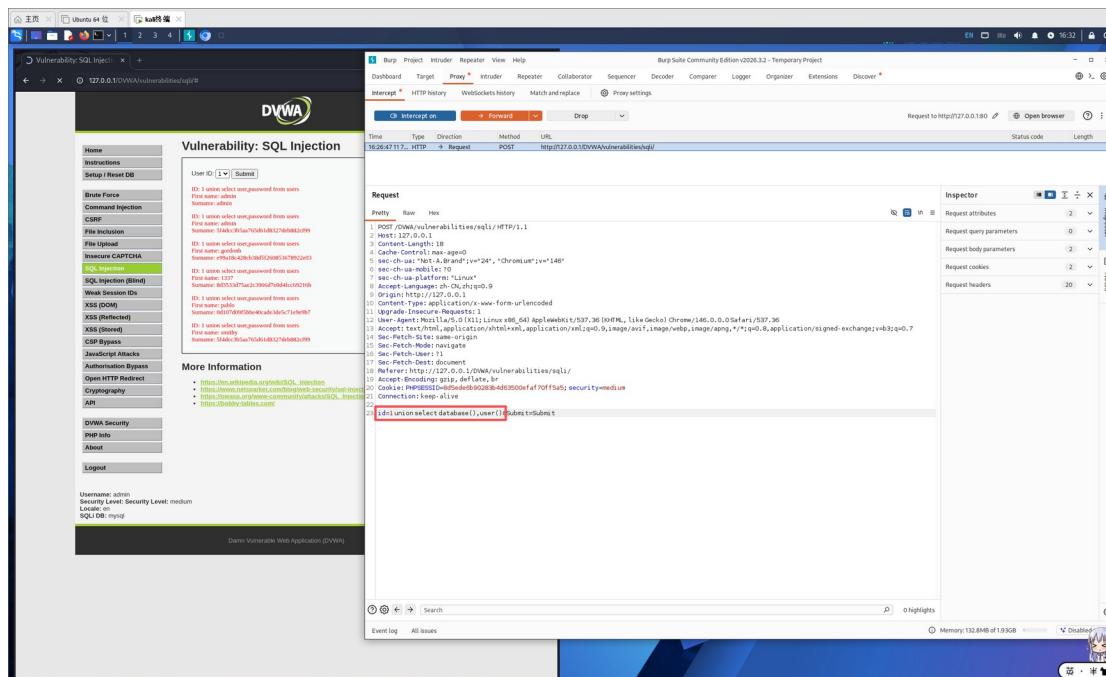
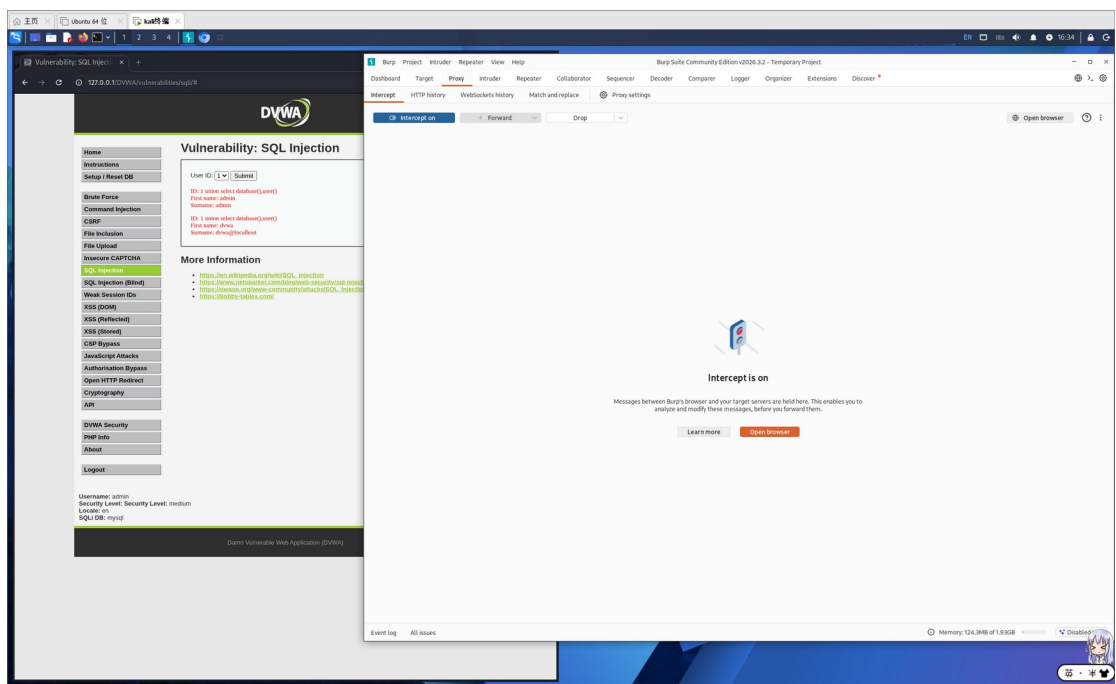


图 6B：成功探测到当前数据库为 dvwa，连接用户为 dvwa@localhost



第七步：Low 与 Medium 级别核心差异对比分析

对比维度	Low 级别	Medium 级别
前端输入方式	文本输入框，允许输入任意字符	下拉菜单，限制只能选择 1-5
后端防护措施	无任何过滤或转义，直接拼接用户输入	使用 <code>mysqli_real_escape_string()</code> 对单引号进行转义，但仍存在注入
注入方式	字符型注入，需闭合单引号并注释掉多余代码	数字型注入，无需闭合单引号，直接拼接逻辑运算
典型 Payload	<code>1' and 1=1 --</code>	<code>1 and 1=1</code>
是否需要 Burp	不需要，直接在浏览器输入框中输入 Payload	必须使用，用于绕过前端下拉菜单限制

Suite		
注入效果	成功拖出所有用户名和密码哈希	成功拖出所有用户名和密码哈希

总结与反思

本次项目在 DVWA 靶场环境中完整复现了 SQL 注入漏洞的发现、验证、利用及信息枚举全过程，覆盖了 Low 和 Medium 两个安全级别。通过本次实战，我得出以下几点结论和反思。

第一，SQL 注入的成因是用户输入被拼接到 SQL 语句中执行，攻击者通过构造特定输入改变 SQL 语句的原本逻辑。Low 级别直接拼接用户输入，未做任何过滤，是最典型的 SQL 注入场景。Medium 级别使用了 `mysqli_real_escape_string` 函数对单引号进行转义，并将前端输入方式改为下拉菜单，但这两项防御均被成功绕过。下拉菜单作为前端验证机制无法抵御中间人攻击，攻击者通过 Burp Suite 拦截并修改 HTTP 请求即可绕过。而 `mysqli_real_escape_string` 只能转义单引号，对于不带单引号的数字型注入 Payload 完全无效。

第二，前端验证不可信，任何安全过滤必须在服务器端实现。真正的防御措施是使用参数化查询，将 SQL 代码与用户输入彻底分离，使数据库能够区分代码和数据，从而防止注入。

第三，通过本次实战我掌握了 SQL 注入的核心利用手法，包括 UNION 查询、`information_schema` 枚举、Burp Suite 抓包改包等关键技术。同时我也认识到，理解漏洞的原理比记住 Payload 更重要，只有理解原理才能在不同的防御措施下找到对应的绕过方法。

后续我计划继续完成 DVWA 中的 XSS 模块和文件上传模块，进一步巩固 Web 安全基础。同时深入学习参数化查询的具体实现方式，从攻击者视角转向防御者视角，做到攻防兼备